

SMART SIEM

Rapport d'Audit Technique

Vérification de conformité avec le cahier des charges

Date de l'audit : 28 June 2026
Branche auditée : data/S3 (intégration complète)
Auteur : andreonana

1. Résumé Exécutif

Cet audit a été réalisé sur la branche **data/S3** du projet Smart SIEM après intégration des travaux des branches backend/S1, infrastructure/S1 et security/S1. L'objectif est de vérifier la conformité du code livré par rapport au cahier des charges sur les 3 semaines de développement.

Module	Avancement	Statut CDC
Pipeline ingestion → Elasticsearch	100%	OK
Agent Syslog (receiver + forwarder)	100%	OK
Module DATA (recherche, rapports, conformité)	100%	OK
RBAC / Authentification JWT	70%	PARTIEL
Corrélation d'événements	0%	MANQUANT
Système d'alertes	0%	MANQUANT
Playbooks SOAR	0%	MANQUANT
UEBA (analyse comportementale)	0%	MANQUANT
Dashboard / Visualisation	0%	MANQUANT
Frontend	0%	MANQUANT
TLS sur les communications	0%	MANQUANT

Conclusion globale : environ **35% du cahier des charges est implémenté**. Le pipeline d'ingestion de logs (collecte → normalisation → stockage Elasticsearch) est pleinement opérationnel et testé. Les fonctionnalités de Semaine 2 (corrélation, alertes, SOAR) et de Semaine 3 (dashboard, frontend) sont absentes ou vides.

2. Semaine 1 — Infrastructure & Fondations

Objectif CDC : "Notre système peut-il recevoir des logs, les normaliser et les stocker de façon sécurisée ?"

Exigence	Statut	Fichier / Détail
Agent Linux déployé et connecté	OK	agents/syslog/syslog_receiver.py + forwarder.py
POST /logs/ingest opérationnel	OK	backend/app/api/v1/routers/logs.py
Endpoint de santé /health	OK	backend/app/api/v1/routers/health.py → {status: ok}
Base de données initialisée	OK	dataset/app/db/init_indices.py — 4 index ES + ILM
1000+ logs simulés ingérés	OK	dataset/scripts/ — 83/83 tests passent
TLS sur toutes les communications	MANQUANT	Absent du docker-compose.yml et des agents
RBAC — première version	PARTIEL	auth.py + roles.py codés, login retourne HTTP 501
README technique d'installation	PARTIEL	README.md existe mais contenu non vérifié

2.1 Ce qui fonctionne

- Agent Syslog : réception UDP/TCP port 514, mise en file d'attente thread-safe, envoi par batch vers le backend.
- Pipeline complet : log brut → SyslogRFC3164Parser → NormalizedLog → Elasticsearch (async).
- Parsers interchangeables : architecture AbstractBaseClass, SyslogParser et JSONLogParser opérationnels.
- Module DATA : 83/83 tests pytest — recherche multi-critères, timeline, export CSV/XLSX, rapports PDF, conformité RGPD.

2.2 Problèmes identifiés

- **TLS absent** : docker-compose.yml ne configure pas de certificats. Les communications agents → backend et backend → ES sont en clair.
- **Login non implémenté** : POST /api/auth/login retourne HTTP 501 (Not Implemented). La source de vérité des utilisateurs n'est pas définie (pas de base users alimentée).

3. Semaine 2 — Intelligence & Détection

Objectif CDC : "Notre système peut-il détecter automatiquement une tentative d'attaque et alerter les bonnes personnes ?"

Exigence	Statut	Fichier / Détail
5+ règles de corrélation documentées	MANQUANT	backend/app/modules/correlation/ — tous vides (0 lignes)
Alertes en temps réel (email/webhook)	MANQUANT	backend/app/modules/alerting/ — tous vides
3 playbooks SOAR	MANQUANT	backend/app/modules/soar/ — tous vides
Interface de recherche multi-critères	OK	dataset/app/db/search.py — IP, type, sévérité, dates, full-text
Résultats horodatés et paginés	OK	search_logs() : tri DESC timestamp, page/page_size
Scénarios d'attaque simulés (MITRE)	OK	dataset/scenarios/ — 6 scénarios T1110, T1048, T1021, T1078
Rapport de tests détection	OK	dataset/tests/ — 83 tests pytest (search, timeline, compliance)

3.1 Modules entièrement vides

Module	Fichiers vides
backend/app/modules/Corrélation/	engine.py, aggregator.py, rules.py, mitre.py
backend/app/modules/Alerting/	alert_manager.py, notifier.py, escalation.py
backend/app/modules/SOAR/	playbook_engine.py, actions.py, triggers.py, integrations.py
backend/app/modules/UEBA/	anomaly_detector.py, baseline.py, behavior_analyzer.py, risk_scorer.py
backend/app/modules/Search avancée/	query_engine.py, pivot.py, timeline.py

4. Semaine 3 — Livrable Final

Objectif CDC : "Démontrer que le Smart SIEM est un système complet, sécurisé et documenté."

Exigence	Statut	Fichier / Détail
Visualisation / Dashboard interactif	MANQUANT	dashboard/ vide — GET /api/dashboard retourne {zeros}
Export rapports PDF et CSV	OK	dataset/app/reports/export.py + pdf_generator.py
Rapports automatiques planifiés	OK	dataset/app/reports/scheduler.py (APScheduler)
Conformité RGPD / rétention	OK	dataset/app/reports/compliance.py + retention.py
Interface frontend utilisable	MANQUANT	frontend/ — package.json et vite.config.js vides
Démonstration système complet	PARTIEL	Pipeline ingestion OK, détection/dashboard absents
Documentation architecture	PARTIEL	docs/schemas-bdd/ présent, docs/api/ non vérifié

5. Bugs Bloquants — Identifiés et Corrigés

Les 4 bugs suivants ont été identifiés lors de l'audit et corrigés dans la foulée. Ils empêchaient le démarrage du backend FastAPI.

Bug 1 — `read_service.py` — Typo `matxh_all`

Impact : Toute requête GET `/api/v1/logs` plantait avec une erreur 400 Elasticsearch.

Avant :

```
query={"matxh_all": {}}
```

Après :

```
query={"match_all": {}}
```

Bug 2 — `json_parser.py` — Méthode `parse_dict()` inaccessible

Impact : La normalisation JSON était cassée — `ingest/json` ne fonctionnait pas.

Avant :

```
def parse_dict(self, ...) [indentée DANS parse()]
```

Après :

```
def parse_dict(self, ...) [méthode de classe correctement indentée]
```

Bug 3 — `schemas/log.py` — Classe `RawLogIngestJSON` manquante

Impact : `logs.py` importait `RawLogIngestJSON` qui n'existait pas → `ImportError` au démarrage.

Avant :

```
(classe absente du fichier)
```

Après :

```
class RawLogIngestJSON(BaseModel) ajoutée avec champ raw_json: dict
```

Bug 4 — `roles.py` — Import relatif incorrect

Impact : `ModuleNotFoundError` au chargement de tous les routers utilisant `require_role()`.

Avant :

```
from auth import decode_access_token, bearer_scheme
```

Après :

```
from app.modules.rbac.auth import decode_access_token, bearer_scheme
```

6. Inventaire Complet des Fichiers

6.1 Backend — Routers API

Fichier	Endpoints	Statut
health.py	GET /api/health	OK
logs.py	POST /ingest, /ingest/json, /ingest/bulk GET / , /{id}	OK
search.py	POST /api/search	OK
audit.py	GET /api/audit	OK
auth.py	POST /api/auth/login (501) POST /api/auth/logout (OK)	PARTIEL
alerts.py	GET /api/alerts → [] GET /api/alerts/{id} → 404	PARTIEL
dashboard.py	GET /api/dashboard → {zeros}	PARTIEL
investigation.py	GET /api/investigation/{id} → {timeline:[]}	PARTIEL
reports.py	GET /api/reports → [] POST /generate → 202	PARTIEL
rules.py	GET/POST /api/rules → stubs	PARTIEL
soar.py	GET /api/soar/playbooks → []	PARTIEL
users.py	GET/POST /api/users → stubs	PARTIEL

6.2 Backend — Modules

Module	Fichiers	Statut
ingestion/	service.py, read_service.py	OK
normalisation/	service.py, tagging.py, parsers/ (base, syslog, json, registry)	OK
rbac/	auth.py, roles.py, local_protection.py, retention.py	OK
correlation/	engine.py, aggregator.py, rules.py, mitre.py — 0 ligne	MANQUANT
alerting/	alert_manager.py, notifier.py, escalation.py — 0 ligne	MANQUANT
soar/	playbook_engine.py, actions.py, triggers.py — 0 ligne	MANQUANT
ueba/	anomaly_detector.py, baseline.py, risk_scorer.py — 0 ligne	MANQUANT
search/	query_engine.py, pivot.py, timeline.py — 0 ligne	MANQUANT
dashboard/	report_generator.py, exporter.py, widgets.py — 0 ligne	MANQUANT

6.3 Dataset — Module DATA

Le module DATA est le plus avancé du projet. 83/83 tests passent.

Fichier	Fonctionnalité	Statut
app/db/elasticsearch_client.py	Connexion ES centralisée	OK
app/db/init_indices.py	Création index + politique ILM	OK
app/db/search.py	search_logs() + get_timeline()	OK
app/db/aggregations.py	count_by_severity, top_ips, logs_per_hour	OK
app/reports/export.py	Export CSV + XLSX horodaté	OK
app/reports/pdf_generator.py	Rapport PDF exécutif (reportlab)	OK
app/reports/compliance.py	Intégrité SHA-256, rétention RGPD	OK
app/reports/scheduler.py	Rapports auto quotidien/hebdo	OK
app/interface/server.py	API FastAPI démo + UI HTML/JS	OK
generators/log_generator.py	1000+ logs réalistes Faker	OK
generators/attack_simulator.py	Scénarios brute-force, exfiltration	OK
scenarios/mitre-attack/	T1110, T1048, T1021, T1078	OK
tests/ (7 fichiers)	83/83 tests pytest	OK

7. Priorités de Développement Recommandées

Classement des travaux restants par priorité décroissante pour atteindre la conformité complète avec le cahier des charges.

P0 — Critique (bloque la démonstration)

- Implémenter le moteur de corrélation (backend/app/modules/correlation/engine.py) — minimum 5 règles (brute-force, port scan, exfiltration, accès inhabituels, multi-sources).
- Implémenter l>alerting (alert_manager.py + notifier.py) — génération d'alertes et envoi email/webhook.
- Brancher auth_router dans main.py et connecter le login à l'index siem-users Elasticsearch.

P1 — Haute valeur (semaine 2 CDC)

- Implémenter 3 playbooks SOAR : blocage IP (iptables/firewall API), désactivation compte, escalade.
- Implémenter le dashboard (GET /api/dashboard) avec données réelles via aggregations.py du module DATA.
- Activer TLS sur docker-compose : certificats auto-signés pour ES et le backend.

P2 — Complémentaire (semaine 3 CDC)

- Frontend : remplir package.json, vite.config.js et implémenter les composants React (DashboardPage, AlertsPage, SearchPage).
- UEBA : baseline comportementale + détection d'anomalies (connexion hors horaires, pays inhabituel).
- Agents supplémentaires : api-collector et filebeat/collector (actuellement vides).

8. Architecture Réelle (État Actuel)

Le schéma suivant représente le flux de données tel qu'il fonctionne réellement au moment de cet audit. Les composants en grisé sont définis dans le code mais vides.

Couche	Composant	État	Flux
Collecte	Agent Syslog UDP/TCP 514	OK	→ POST /ingest/bulk
Collecte	Agent API Collector	MANQUANT	(non implémenté)
Collecte	Filebeat → Forwarder	PARTIEL	Config présente, intégration partielle
Ingestion	POST /api/v1/logs/ingest	OK	→ normalisation → ES
Normalisation	SyslogParser RFC3164	OK	→ ParsedLog → NormalizedLog
Normalisation	JSONLogParser	OK	→ ParsedLog → NormalizedLog
Stockage	Elasticsearch — index logs-siem	OK	Mappings typés, ILM 30 jours
Recherche	search_logs() + get_timeline()	OK	Multi-critères, CIDR, full-text
Corrélation	correlation/engine.py	MANQUANT	Vide — aucune règle active
Alertes	alerting/alert_manager.py	MANQUANT	Vide — aucune alerte générée
SOAR	soar/playbook_engine.py	MANQUANT	Vide — aucun playbook
Reporting	PDF + CSV + conformité RGPD	OK	Scheduler automatique
Frontend	React/Vite	MANQUANT	Fichiers vides

9. Conclusion

Le projet Smart SIEM dispose d'une fondation technique solide : le pipeline d'ingestion de logs est complet, testé et opérationnel. Le module DATA répond à l'intégralité des exigences de stockage, recherche et conformité du cahier des charges.

Cependant, les fonctionnalités qui font la valeur d'un SIEM — la **corrélation d'événements**, la **génération d'alertes** et les **playbooks de réponse automatique** — sont absentes. Sans elles, le système collecte et stocke des logs mais ne détecte aucune menace, ce qui ne satisfait pas l'objectif central du cahier des charges de la Semaine 2.

Les 4 bugs bloquants qui empêchaient le démarrage du backend ont été corrigés dans le cadre de cet audit (typo Elasticsearch, indentation json_parser, classe Pydantic manquante, import relatif incorrect). Le backend FastAPI démarre désormais correctement.

Semaine	Conformité CDC	Remarque
Semaine 1	70%	Pipeline OK, TLS et login manquants
Semaine 2	20%	Recherche OK, corrélation/alertes/SOAR absents
Semaine 3	30%	Reporting OK, dashboard et frontend absents
Global	~35%	Fondation solide, intelligence SIEM à implémenter